



电子科技大学

University of Electronic Science and Technology of China

信息与软件工程学院

SCHOOL OF INFORMATION AND SOFTWARE ENGINEERING

W1D3-C语言知识强化：数组与结构

Linux下Web Server的设计

讲授内容

- 训练目的
- 训练原理
- 训练内容



- 训练目的
 - 回顾二叉树数据结构
 - 实现基于二叉树的各种操作
 - ✓ 查找
 - ✓ 创建
 - ✓ 删除
 - ✓ 修改
 - ✓ 增加
 - 利用二叉树实现地址簿

二叉树定义

定义:把满足以下两个条件的树结构称为**二叉树**(Binary Tree) :

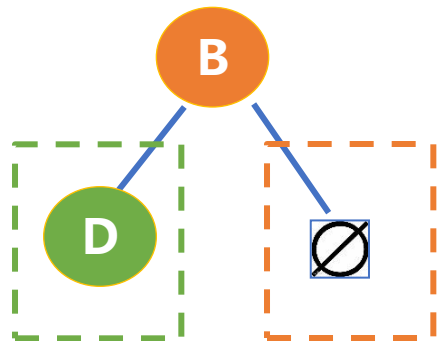
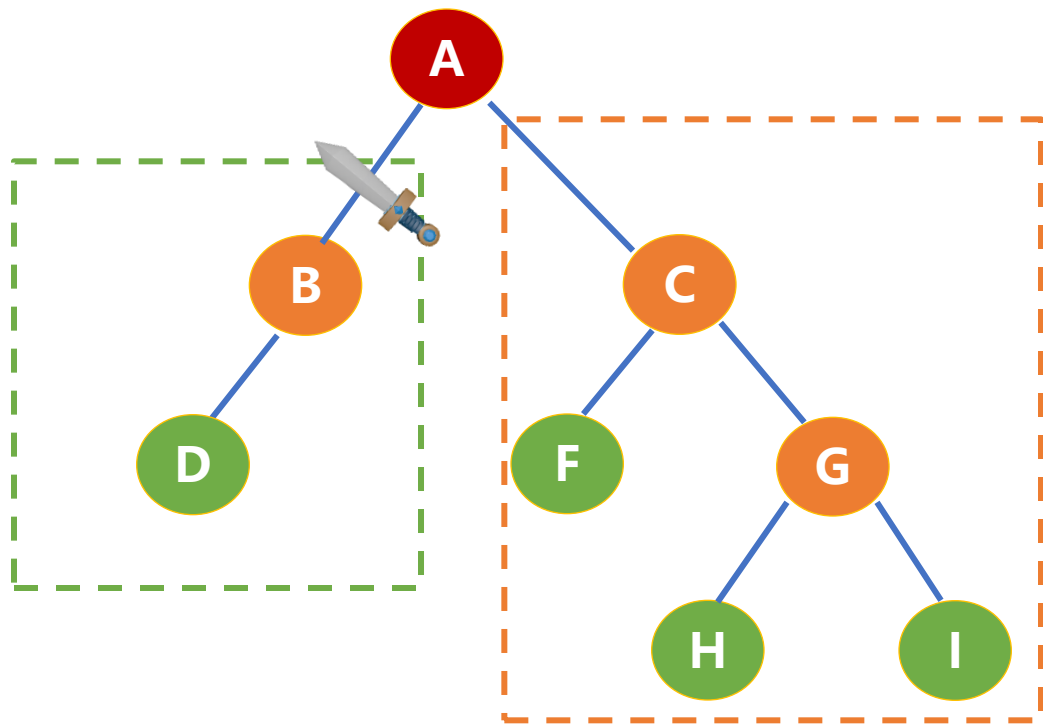
- ①每个结点的度都不大于2。
- ②每个结点的孩子结点次序不能任意颠倒。

一个二叉树的每个结点可能含有0、1或2个孩子，且每个孩子有左右之分。

有序



二叉树是满足递归定义的数据结构



二叉树几个性质

🌳 **性质1**：在二叉树的第 i 层上至多有 2^{i-1} 个结点($i \geq 1$)。

🌳 证明：

🌳 当 $i=1$ 时，整个二叉树只有一根结点，此时 $2^{i-1}=2^0=1$ ，结论成立。

🌳 假设 $i=k$ 时结论成立，即第 k 层上结点总数最多为 2^{k-1} 个。

🌳 现证明当 $i=k+1$ 时，结论成立：

- 因为二叉树中每个结点的度最大为2，则第 $k+1$ 层的结点总数最多为第 k 层上结点最大数的2倍，即 $2 \times 2^{k-1} = 2^{(k+1)-1}$ ，故结论成立。



二叉树几个性质

♣ 性质2: 深度为k的二叉树至多有 2^k-1 个结点 ($k \geq 1$)

♣ 证明: 因为深度为k的二叉树, 其结点总数的最大值是将二叉树每层上结点的最大值相加, 所以深度为k的二叉树的结点总数至多为:

$$\text{♣ } \sum_{i=1}^k \text{第} i \text{层上的最大结点个数} = \sum_{i=1}^k 2^{i-1} = 2^k - 1$$

$$x = \sum_{i=1}^k 2^{i-1} = 2^0 + 2^1 + \dots + 2^{k-1}$$

$$2x = 2 * (2^0 + 2^1 + \dots + 2^{k-1}) = 2^1 + 2^2 + \dots + 2^{k-1} + 2^k = 2^0 + 2^1 + \dots + 2^{k-1} + 2^k - 2^0$$

$$x = 2^k - 1$$

♣ 故结论成立。



二叉树几个性质

 **性质3**：对任意一棵二叉树T，若终端结点数为 n_0 ，而其度数为2的结点数为 n_2 ，则：

$$\bullet \quad n_0 = n_2 + 1$$

 **证明**：设二叉树中结点总数为 n ， n_1 为二叉树中度为1的结点总数。因为二叉树中所有结点的度小于等于2，所以有：
$$n = n_0 + n_1 + n_2$$

 设二叉树中分支数目为 B ，因为除根结点外，每个结点均对应一个进入它的分支，所以有：
$$n = B + 1$$

 又因为二叉树中的分支都是由度为1和度为2的结点发出，所以分支数目为：
$$B = n_1 + 2n_2$$

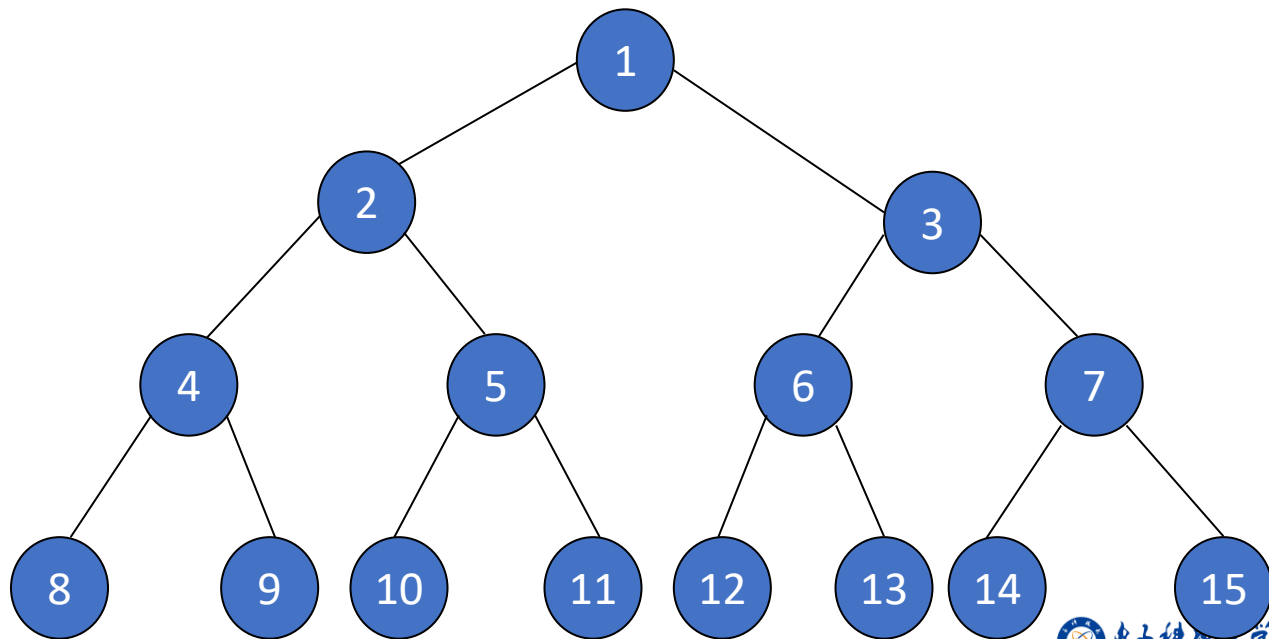
 整理上述两式可得到：
$$n = B + 1 = n_1 + 2n_2 + 1$$

 将 $n = n_0 + n_1 + n_2$ 代入上式得出 $n_0 + n_1 + n_2 = n_1 + 2n_2 + 1$ ，整理后得 $n_0 = n_2 + 1$ ，故结论成立。



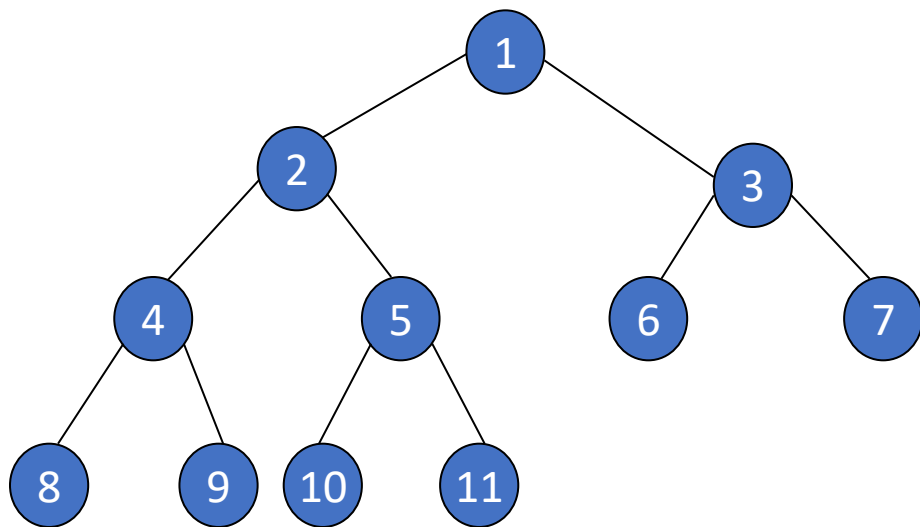
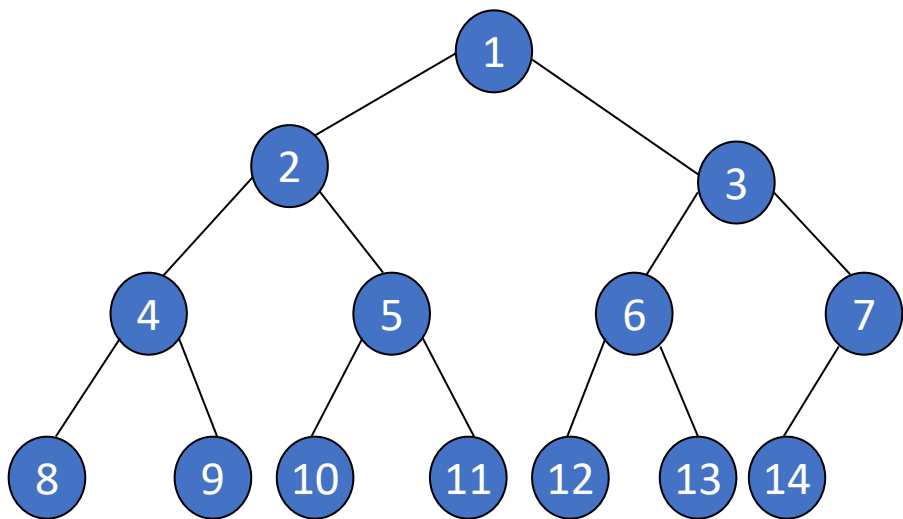
满二叉树

🌳 深度为 k 且有 2^k-1 个结点的二叉树。在满二叉树中，每层结点都是满的，即每层结点都具有最大结点数。



完全二叉树

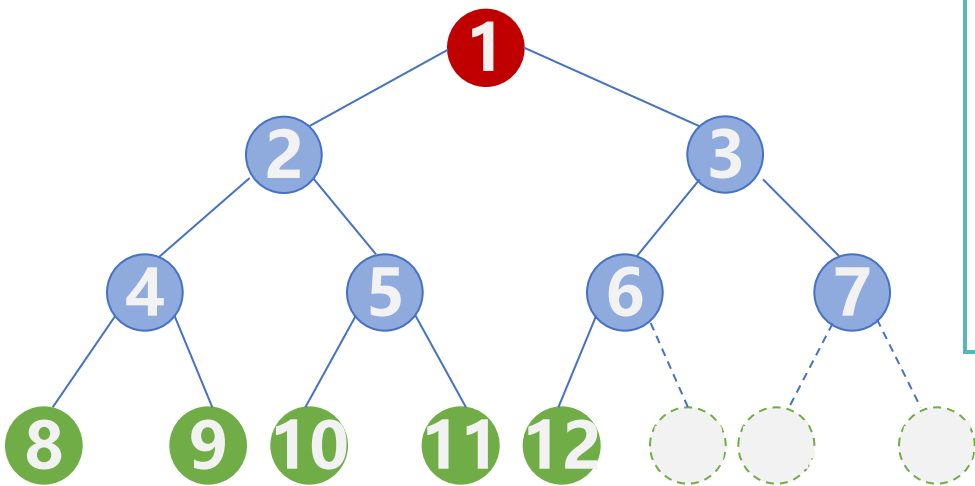
对一棵具有 n 个结点的二叉树 T 按层序编号，如果编号为 i ($1 \leq i \leq n$) 的结点与同样深度的满二叉树中编号为 i 的结点在二叉树中的位置完全相同，则称 T 为完全二叉树。



二叉树的存储结构



二叉树的顺序存储



```
#define MaxSize 100
struct TreeNode{
    ElemType value; // 结点中的数据元素
    bool isEmpty; // 结点是否为空
};
TreeNode t[MaxSize];
```

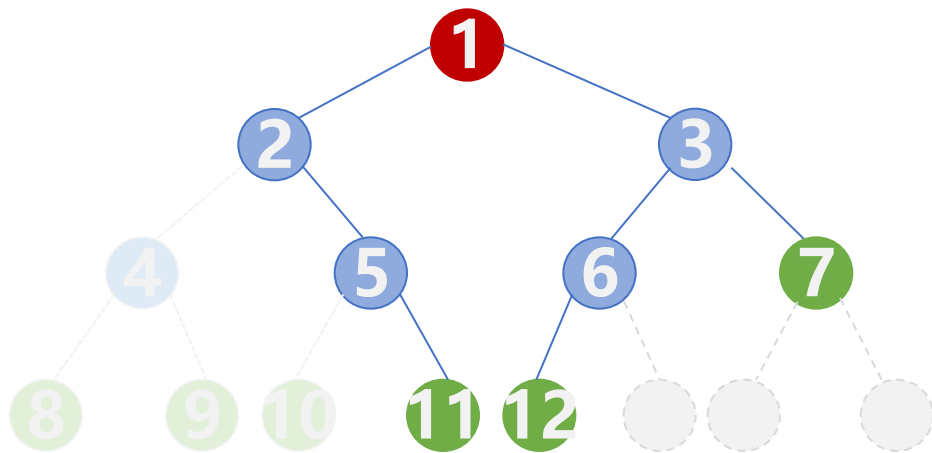
定义一个长度为MaxSize的数组t，按照从上至下、从左至右的顺序依次存储完全二叉树中的各个结点



t[0] t[1] t[2]



二叉树的顺序存储



i

二叉树的顺序存储中，一定要把二叉树的结点编号与完全二叉树对应起来

i的左孩子 —— $2i$
i的右孩子 —— $2i+1$
i的父结点 —— $\lfloor i/2 \rfloor$

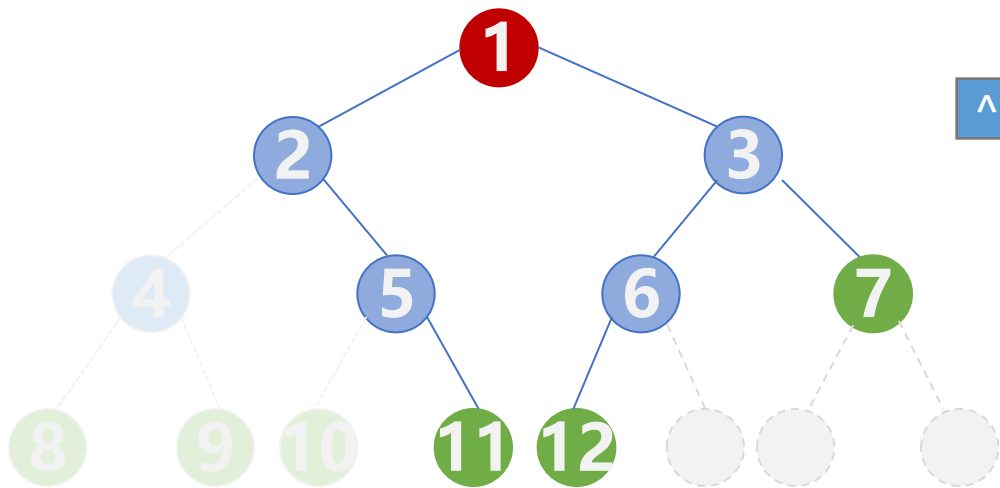


t[0] t[1] t[2]

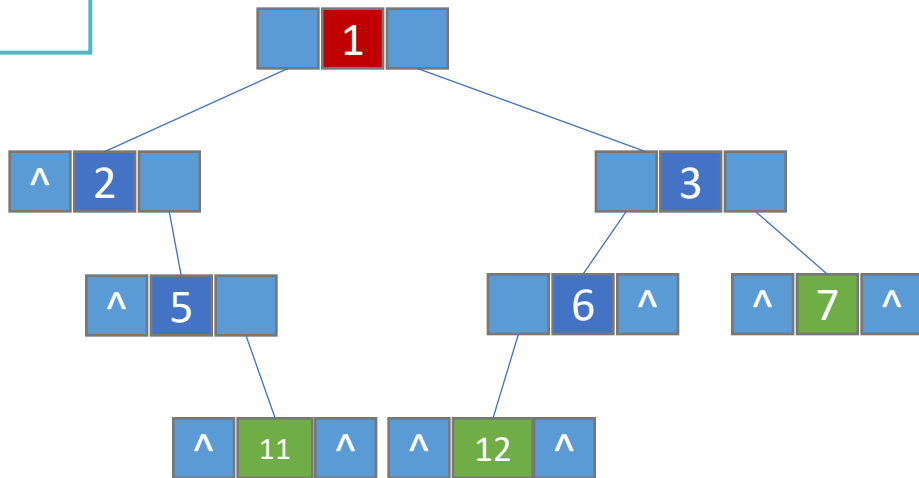


二叉树的链式存储

```
typedef struct Node
{
    DataType data;           //数据域
    struct Node * LChild;    //左孩子指针
    struct Node * RChild;    //右孩子指针
}BiTNode, *BiTree;
```



★ n 个结点的二叉链表共有
 $n+1$ 个空链域

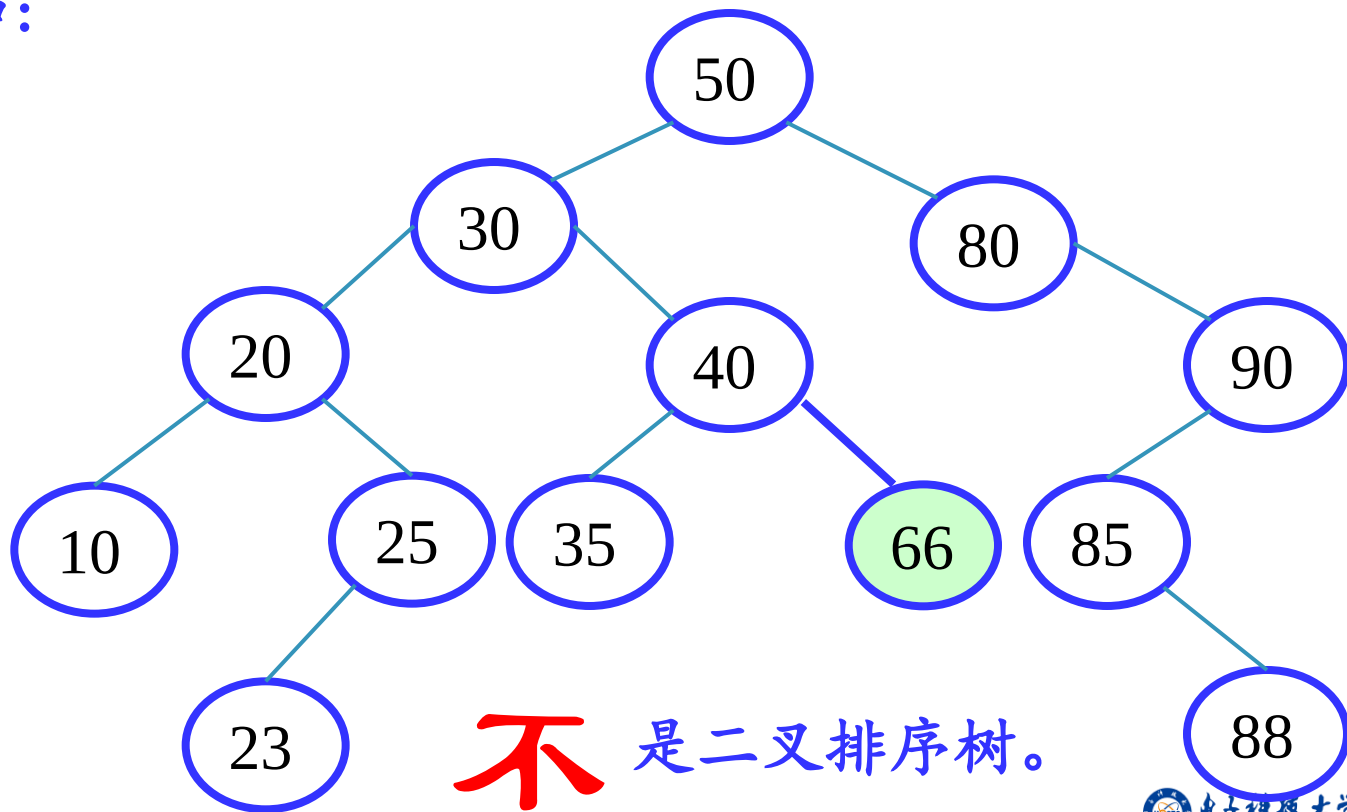


二叉排序树又称二叉查找树(BST,Binary Search Tree)，它是一种特殊的二叉树。其定义为：

- ①若它的左子树非空，则左子树上所有结点的值均小于根结点的值
- ②若它的右子树非空，则右子树上所有结点的值均大于或等于根结点的值
- ③它的左右子树也分别为二叉排序树。

二叉排序树

例如：

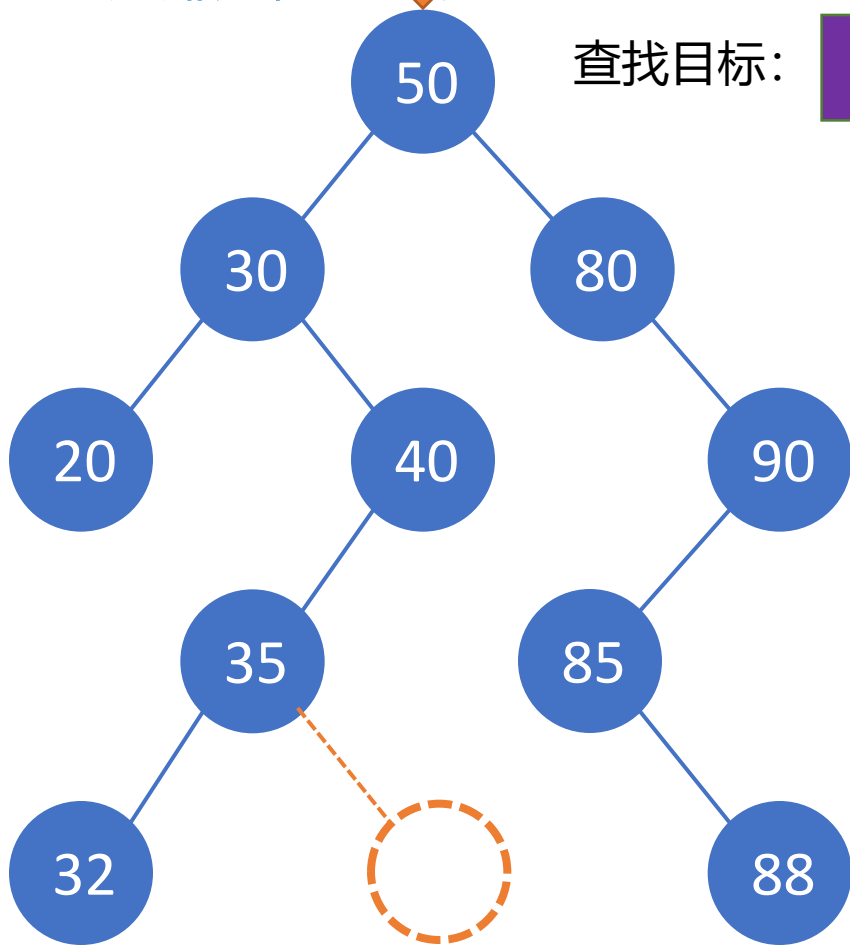


二叉排序树-查找

查找目标:

37

- 二叉排序树可看做是一个有序表，在二叉排序树bt上查找关键字为k的记录，
- 成功时返回该结点指针，否则返回NULL



lchild

key

rchild

```
typedef struct node{
    KeyType key ;           /*关键字的值*/
    struct node *lchild,*rchild; /*左右孩子指针*/
}BSTNode,*BSTree;
```

```
BSTree SearchBST(BSTree bt, KeyType k) {
    if (bt==NULL || bt->key==k) //递归出口
        return bt;
    if (k<bt->key)
        return SearchBST(bt->lchild, k); //在左子树中递归查找
    else
        return SearchBST(bt->rchild, k); //在右子树中递归查找
}
```

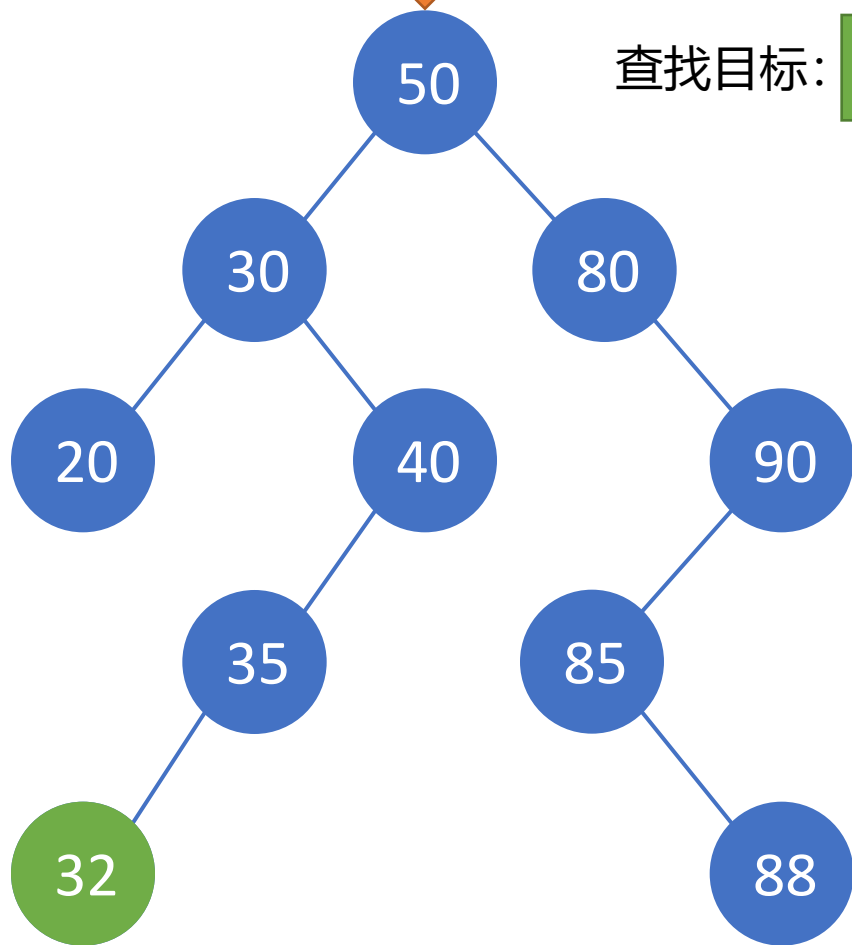


二叉排序树-查找

查找目标:

32

- 二叉排序树可看做是一个有序表，在二叉排序树bt上查找关键字为k的记录，
- 成功时返回该结点指针，否则返回NULL



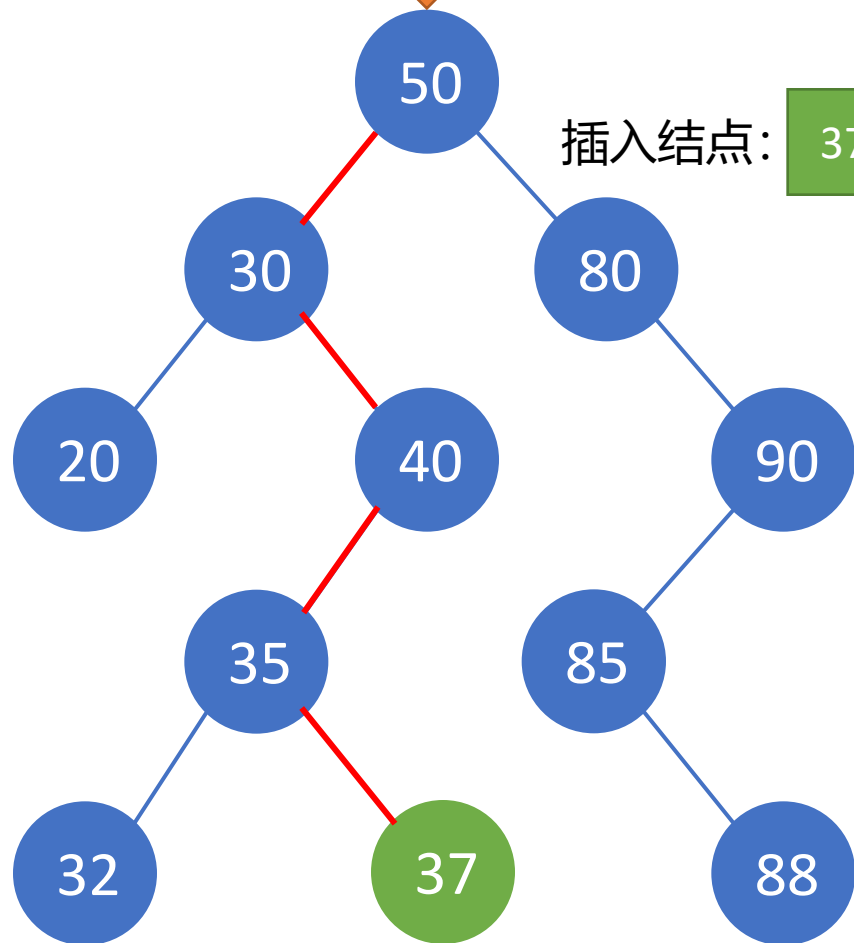
lchild	key	rchild
--------	-----	--------

```
typedef struct node{
    KeyType key ;           /*关键字的值*/
    struct node *lchild,*rchild; /*左右孩子指针*/
}BSTNode,*BSTree;
```

```
BSTree SearchBST(BSTree bt, KeyType k) {
    if (bt==NULL || bt->key==k)           //递归出口
        return bt;
    if (k<bt->key)
        return SearchBST(bt->lchild, k); //在左子树中递归查找
    else
        return SearchBST(bt->rchild, k); //在右子树中递归查找
}
```



二叉排序树-新增



插入结点: 37

若二叉排序树为空，则插入结点应为新的根结点
否则根据排序树的性质继续在其左、右子树上查找

直至某个结点的左子树或右子树为空为止
则插入结点应为该结点的左孩子或右孩子

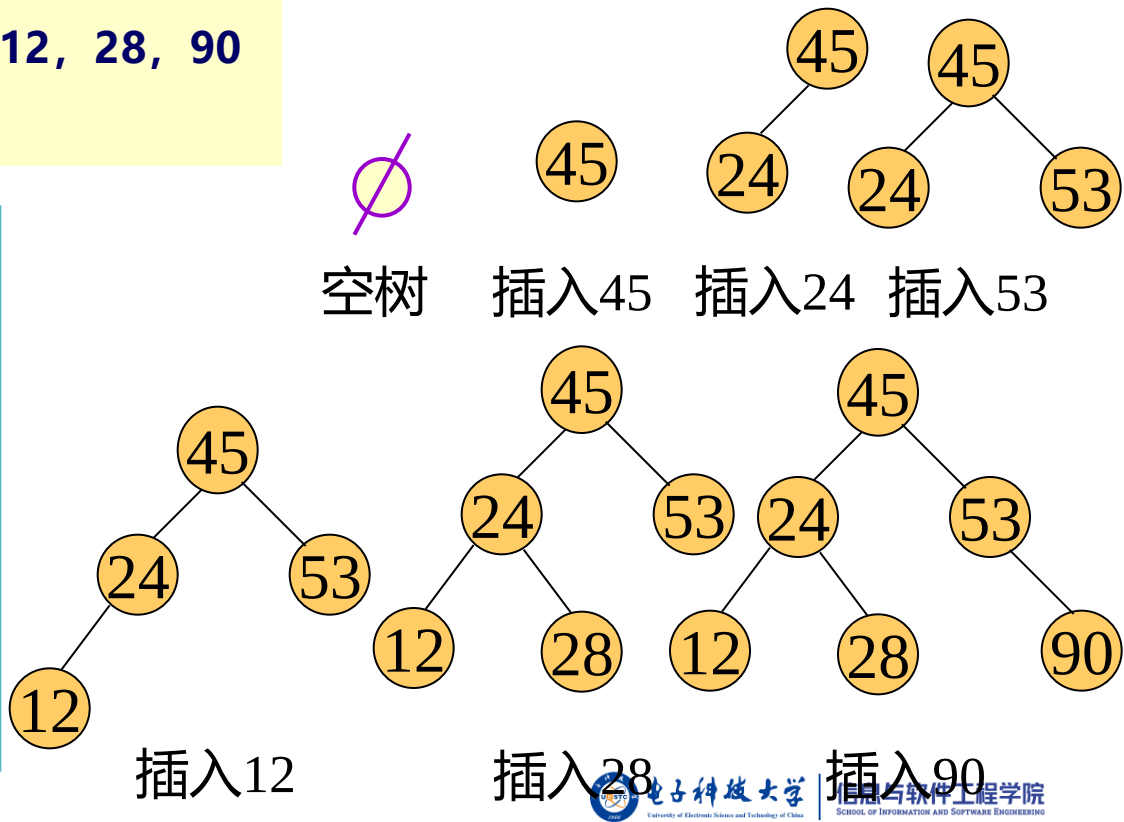
```
void InsertBST(BSTree *bst, KeyType key) {
    /*若在二叉排序树中不存在关键字等于key的元素，插入该元素*/
    BiTree s;
    if (*bst==NULL) {
        /*递归结束条件*/
        s=(BSTree)malloc(sizeof(BSTNode));/*申请新的结点
s*/
        s->key=key;
        s->lchild=NULL;
        s->rchild=NULL;
        *bst=s;
    }
    else if (key < (*bst)->key)
        InsertBST(&((*bst)->lchild), key);/*将s插入左子树
*/
    else if (key > (*bst)->key)
        InsertBST(&((*bst)->rchild), key);/*将s插入右子树
*/
}
```

二叉排序树的创建

设关键字的输入顺序为：45, 24, 53, 12, 28, 90

按算法生成的二叉排序树的过程：

```
void CreateBST(BSTree *bst) {  
    /*从键盘输入元素的值，创建相应的  
    二叉排序树*/  
    KeyType key;  
    *bst=NULL;  
    scanf("%d", &key);  
    while(key!=ENDKEY){/*ENDKEY为自  
    定义常数*/  
        InsertBST(bst, key);  
        scanf("%d", &key);  
    }  
}
```



训练1 二叉排序树练习1

从.csv文件中读取用户（用户名、联系方式等）信息，

1.以用户名为关键字，创建二叉排序树存放信息

2.实现用户信息的查找

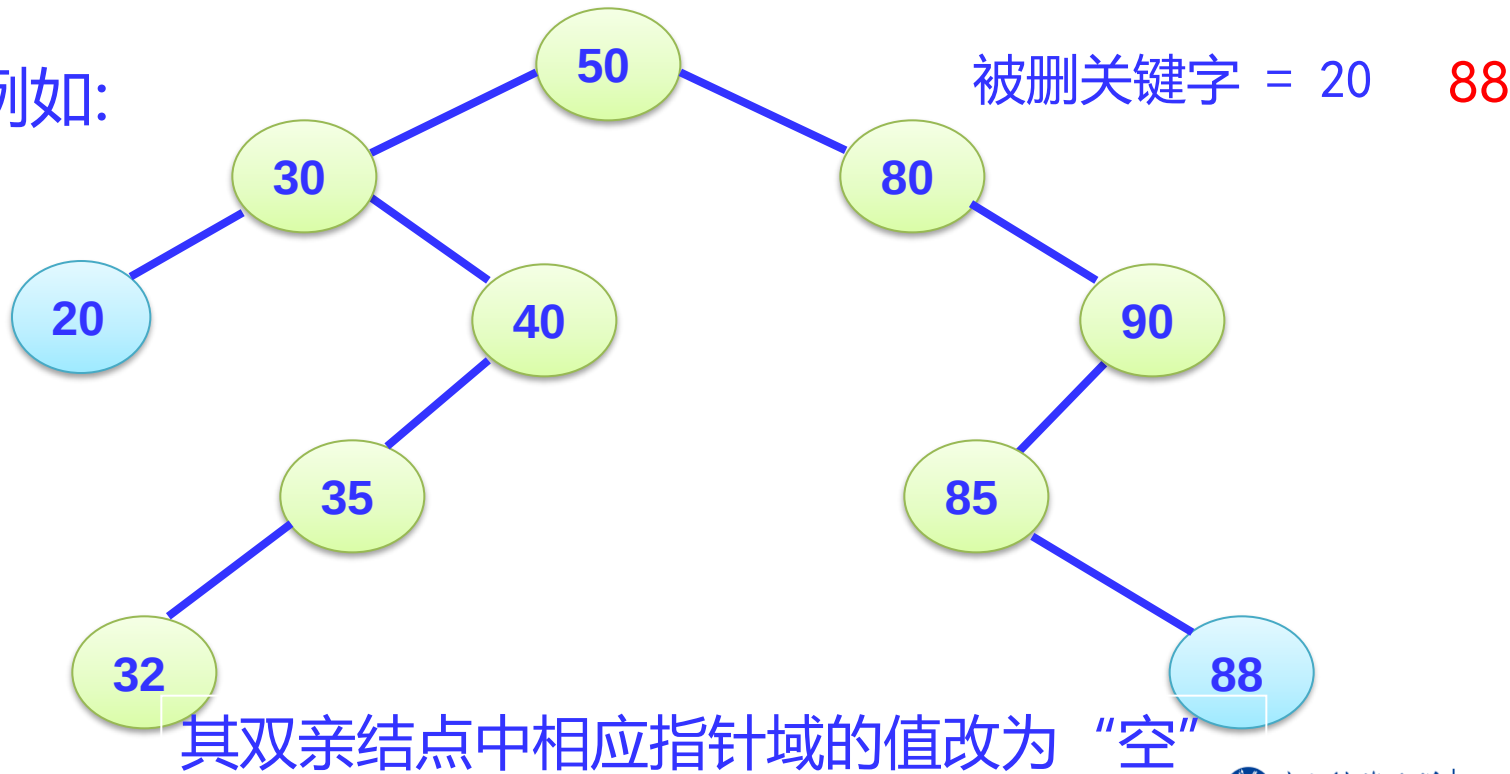
3.实现用户信息的新增



二叉排序树-删除

1) 被删除的结点是叶子结点：直接删去该结点。

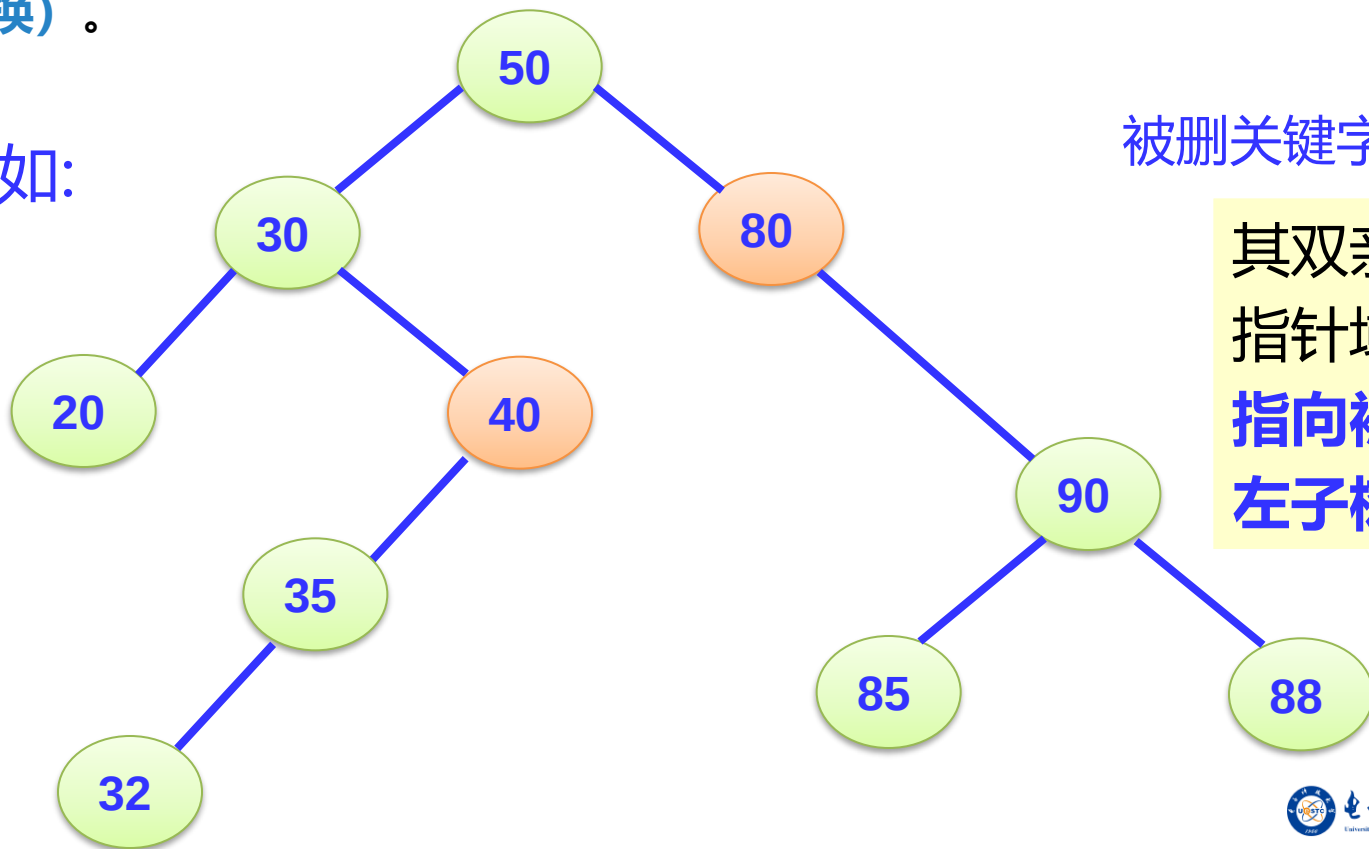
例如：



二叉排序树-删除

2) 被删除的结点**只有左子树**或者**只有右子树**，用其左子树或者右子树**替换它（结点替换）**。

例如:



被删关键字 = 40 80

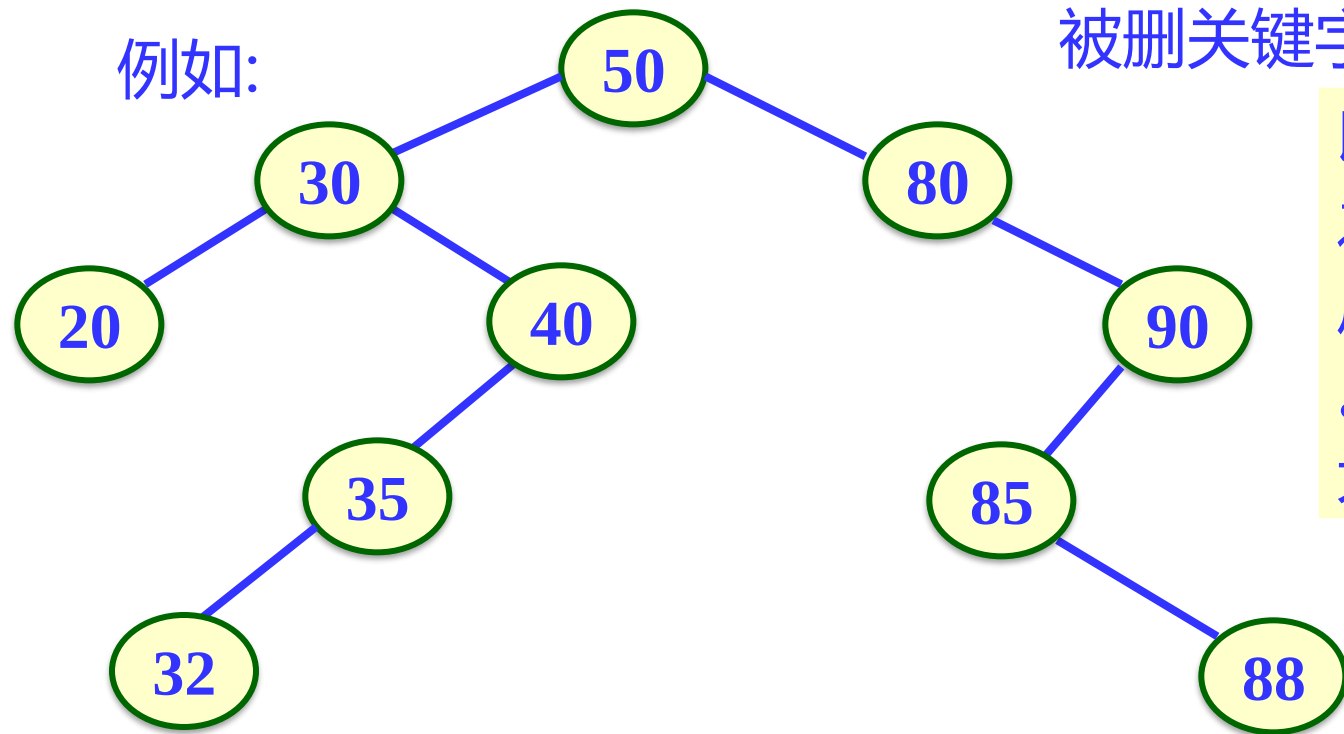
其双亲结点的相应
指针域的值改为：
**指向被删除结点的
左子树或右子树**

二叉排序树-删除

3) 被删除的结点既有左子树，也有右子树

例如:

被删关键字 = 50



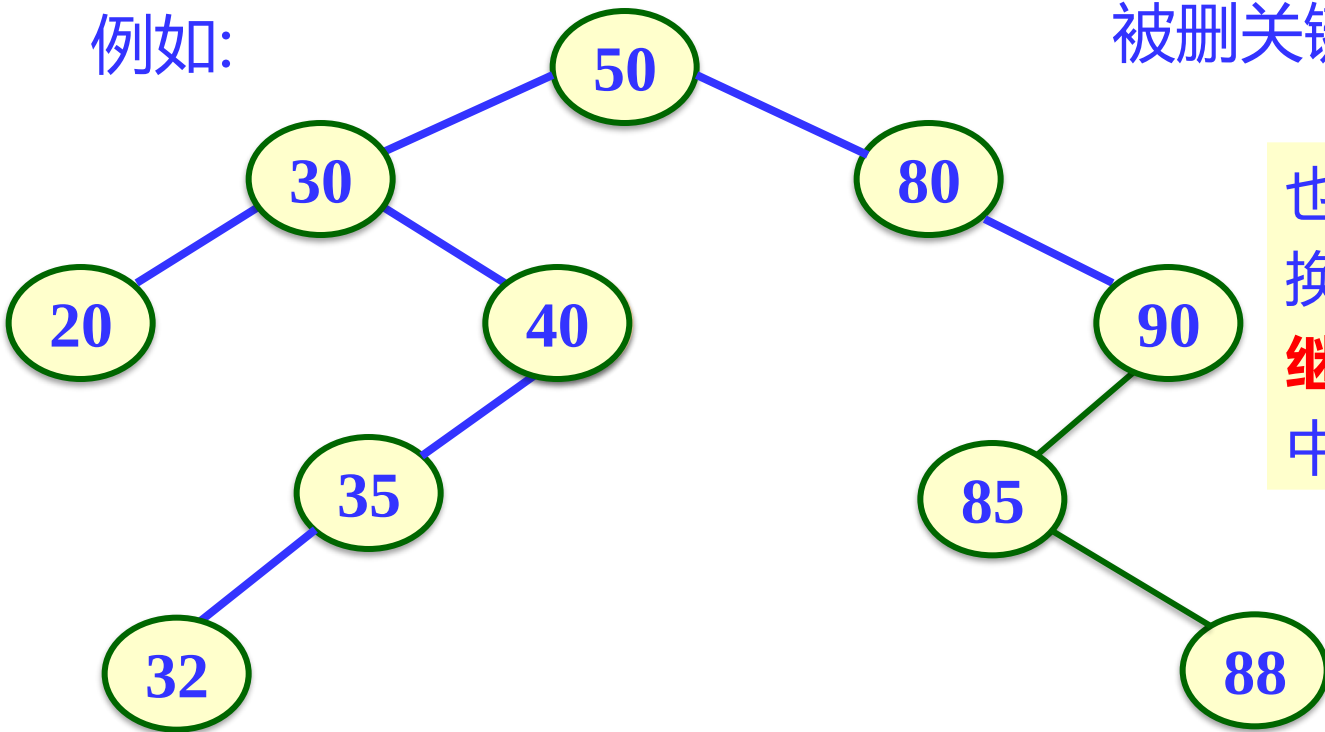
以其**中序前趋值**替换之（值替换），然后再**删除该前趋结点**。前趋是左子树中最大的结点。



二叉排序树-删除

3) 被删除的结点既有左子树，也有右子树

例如:



被删关键字 = 50

也可以用其中**序后继**替换之，然后**再删除该后继结点**。后继是右子树中最小的结点。



训练2 二叉排序树练习2

在训练1创建的二叉排序树的基础上，

- 1.实现用户信息的删除
- 2.实现用户信息的修改
- 3.实现用户信息保存到.csv文件上



参考代码(指导老师酌情给)

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

// 定义联系人结构体
struct Contact {
    char name[20];
    char phone[20];
};

// 定义二叉树节点结构体
struct TreeNode {
    struct Contact contact;
    struct TreeNode *left;
    struct TreeNode *right;
};
```

```
// 创建新节点
struct TreeNode *createNode(struct Contact contact) {
    struct TreeNode *newNode = (struct TreeNode*) malloc(sizeof(struct TreeNode));
    newNode->contact = contact;
    newNode->left = NULL;
    newNode->right = NULL;
    return newNode;
}

// 插入节点
struct TreeNode *insertNode(struct TreeNode *root, struct Contact contact) {
    if (root == NULL) {
        return createNode(contact);
    } else {
        if (strcmp(contact.name, root->contact.name) < 0) {
            root->left = insertNode(root->left, contact);
        } else {
            root->right = insertNode(root->right, contact);
        }
        return root;
    }
}
```



参考代码(指导老师酌情给)

```
// 删除节点
struct TreeNode *deleteNode(struct TreeNode *root, char *name) {
    if (root == NULL) {
        return NULL;
    }
    if (strcmp(name, root->contact.name) < 0) {
        root->left = deleteNode(root->left, name);
    } else if (strcmp(name, root->contact.name) > 0) {
        root->right = deleteNode(root->right, name);
    } else {
        if (root->left == NULL) {
            struct TreeNode *temp = root->right;
            free(root);
            return temp;
        } else if (root->right == NULL) {
            struct TreeNode *temp = root->left;
            free(root);
            return temp;
        }
        struct TreeNode *temp = root->right;
        while (temp->left != NULL) {
            temp = temp->left;
        }
        root->contact = temp->contact;
        root->right = deleteNode(root->right, temp->contact.name);
    }
    return root;
}
```

```
// 查找节点
struct TreeNode *searchNode(struct TreeNode *root, char *name) {
    if (root == NULL || strcmp(root->contact.name, name) == 0) {
        return root;
    } else if (strcmp(name, root->contact.name) < 0) {
        return searchNode(root->left, name);
    } else {
        return searchNode(root->right, name);
    }
}

// 修改联系人信息
void modifyContact(struct TreeNode *root, char *name, char *phone) {
    struct TreeNode *node = searchNode(root, name);
    if (node != NULL) {
        strcpy(node->contact.phone, phone);
        printf("联系人信息已修改\n");
    } else {
        printf("未找到该联系人\n");
    }
}
```



参考代码(指导老师酌情给)

```
// 打印联系人信息
void printContact(struct TreeNode *node) {
    printf("姓名: %s, 电话: %s\n", node->contact.name, node->contact.phone);
}

// 中序遍历
void inorderTraversal(struct TreeNode *root, FILE *fp) {
    if (root != NULL) {
        inorderTraversal(root->left, fp);
        fprintf(fp, "%s,%s\n", root->contact.name, root->contact.phone);
        inorderTraversal(root->right, fp);
    }
}
```

```
int main() {
    struct TreeNode *root = NULL;
    FILE *fp = fopen("contacts.csv", "r"); // 从文件中读取联系人信息
    if (fp == NULL) {
        printf("打开文件失败\n");
        return 1;
    }
    char line[50];
    while (fgets(line, sizeof(line), fp)) {
        char *name = strtok(line, ",");
        char *phone = strtok(NULL, ",");
        struct Contact contact;
        strncpy(contact.name, name, 19);
        strncpy(contact.phone, phone, 19);
        contact.name[19] = '\0';
        contact.phone[19] = '\0';
        root = insertNode(root, contact);
    }
    fclose(fp);
    // 查找联系人
    struct TreeNode *node1 = searchNode(root, "张三");
    if (node1 != NULL) {
        printf("找到联系人: \n");
        printContact(node1);
    } else {
        printf("未找到该联系人\n");
    }
    modifyContact(root, "张三", "111111111"); // 修改联系人信息
    root = deleteNode(root, "李四"); // 删除联系人
    // 将结果保存到文件中
    fp = fopen("contacts_result.csv", "w");
    if (fp == NULL) {
        printf("打开文件失败\n");
        return 1;
    }
    inorderTraversal(root, fp);
    fclose(fp);
    return 0;
}
```

Webserver V0.3版

为用户建立二叉树的快速索引



mini_webserver

工程基础模块 Day1 && Day4

构建子模块 D1
Makefile
编译、清理、调试版本

配置子模块 D1
Config.c
端口、根目录、线程数

日志子模块 D1,D4
log.c
访问日志、错误日志

用户数据模块 Day2 && Day3

用户结构子模块 D2-D3
user.c
链表、二叉树、增删改查

CSV存储子模块 D2
csv_store.c
用户数据加载和保存

索引查找子模块 D3
user_index.c
按用户名快速查找

网络服务模块 Day6 && Day7

TCP监听子模块 D6
server.c
socket, bind, listen

连接管理子模块 D7
client.c
连接与状态、关闭释放

并发处理模块 Day4-5 && Day8-9

进程模型子模块 D4
process.c
父子进程协同

线程同步子模块 D5
thread.c
锁、条件变量、共享资源保护

线程池子模块 D8
thread_pool.c
任务队列、工作线程

事件驱动子模块 D9
event_loop.c
select / epoll

HTTP业务模块 Day10-Day14

请求解析子模块 D10
http_request.c
方法、URL、Header、Body

静态资源子模块 D11
static_file.c
HTML、CSS、图片读取

GET/POST处理子模块 D12
handler.c
表单提交、参数读取

路由配置子模块 D13
router.c
URL匹配、处理函数绑定

认证安全子模块 D14
auth.c
登录校验、权限控制、路径检查

响应生成子模块 D10-D14
http_response.c
状态码、响应头、响应体

测试模块 Day1-15

每日增加功能的相关测试

如: test_day01.sh

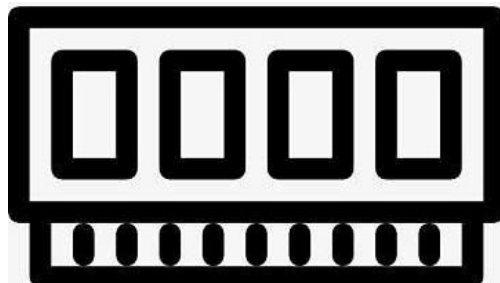
如: bench.c 并发压力测试/响应时间等

V0.2版已经有 user_store 链表，可以读写 data/users.csv。

V0.3的任务：在链表基础上建立一个按用户名查找的索引，让后面的认证模块可以更快判断用户是否存在、密码是否正确。



data/users.csv



user_store
内存中的链表

user_index
内存中二叉查找树

为什么要在之前链表基础上增加二叉查找树？

用户链表

zhangsan -> lisi -> wangwu -> zhaoliu

二叉查找树查找



新增的结构体

```
typedef struct user_index_node {  
    user_t *user;  
    struct user_index_node *left;  
    struct user_index_node *right;  
} user_index_node_t;
```

```
typedef struct {  
    user_index_node_t *root;  
    int size;  
} user_index_t;
```

容易出错：重复free



新增文件包括：

- include/user_index.h
- src/user_index.c
- tests/test_day03.sh
- data/users_large.csv

修改文件包括

- include/user_store.h
- src/user_store.c
- src/main.c
- Makefile
- README.md
- docs/debug-log.txt (可选)

- users index 能按用户名顺序输出
- users find-index zhangsan 能找到用户
- users find-index nobody 输出 NOT_FOUND
- users compare <username> 能输出链表和树的查找步骤
- 释放索引树后程序正常退出
- 能解释 user_index_node_t 中为什么保存 user_t *